

INVEC

Documentacao Tecnica

Arquitetura, seguranca, banco de dados e endpoints

Pontual Tecnologia
v1.0 | 2026

Visao Geral da Arquitetura

O Invec e um sistema cliente-servidor composto por dois componentes independentes: um **backend FastAPI** (Python) que roda como servico Windows e acessa o banco Firebird do Automec, e um **app Android** (Kotlin) que se comunica com o backend via HTTP/JSON pelo Wi-Fi interno da empresa.

```
[Celulares Android] <--HTTP/JSON/Wi-Fi--> [InvecServidor.exe / Windows Service]
                                     |
                                     [Firebird 5 / Banco Automec]
```

- Varios celulares operam simultaneamente no mesmo deposito
- Servidor registrado como servico Windows (InvecAPI via NSSM) - inicia automaticamente com o Windows
- Toda comunicacao e autenticada via JWT Bearer token no header Authorization
- Operacoes de escrita no banco usam UPDATE ... RETURNING atomico para evitar race conditions

Stack Tecnologica

Camada	Tecnologia	Versao / Detalhe
Backend linguagem	Python	3.13
Backend framework	FastAPI + Uvicorn	FastAPI 0.115+, Uvicorn ASGI
Banco de dados	Firebird 5 via firebird-driver	Driver nativo Python, sem ODBC
Distribuicao servidor	PyInstaller + NSSM	Single-file EXE + servico Windows
App linguagem	Kotlin	JVM target 17
App SDK minimo	Android 8.0 (API 26)	Target API 35
HTTP client app	Retrofit2 + OkHttp3	Retrofit 2.11, OkHttp 4.x
Camera / barcode	CameraX + ML Kit Barcode Scanning	CameraX 1.3+, ML Kit offline
UI app	Material Design 3 + ViewBinding	Material 1.12+
Autenticacao	JWT HS256 (sessao)	python-jose 3.3+
Licenca	JWT RS256 / RSA 2048-bit	cryptography 41+
Validacao dados backend	Pydantic v2	via FastAPI

Estrutura do Repositorio (Monorepo)

```
inventario-app/
  api/                # backend Python
    main.py           # FastAPI app + lifespan (licenca + migrations)
    server.py         # entrypoint Uvicorn para PyInstaller
    instalador.py     # GUI Tkinter do instalador Windows
    servidor.spec     # PyInstaller spec do servidor
```

```

instalador.spec          # PyInstaller spec do instalador
requirements.txt
app/
  database.py            # get_connection() context manager Firebird
  security.py            # JWT HS256, get_current_user(), roles
  licenca.py             # validacao RSA 2048-bit (chave publica embutida)
  migrations.py          # DDL idempotente executado no startup
  models/schemas.py     # modelos Pydantic (request + response)
  routers/
    auth.py              # login, rate limit, usuarios mobile
    depositos.py         # listagem de depositos
    produtos.py          # busca por codigo de barras / descricao
    inventario.py        # bipagem, relatorio, consolidacao, auditoria
    operadores.py        # CRUD de operadores fisicos
app/                      # Android Kotlin
src/main/java/br/com/inventario/
  ui/base/TimeoutActivity.kt
  ui/login/LoginActivity.kt
  ui/main/MainActivity.kt
  ui/scanner/ScannerActivity.kt
  ui/relatorio/RelatorioActivity.kt
  ui/recontagem/RecontagemActivity.kt
  ui/historico/HistoricoActivity.kt
  ui/auditoria/AuditoriaActivity.kt
  ui/operadores/OperadoresActivity.kt
  ui/usuarios/UsuariosActivity.kt
  data/api/ApiService.kt    # interface Retrofit com todos os endpoints
  data/api/RetrofitClient.kt # OkHttp + interceptor Bearer + interceptor 401
  data/model/Models.kt      # data classes request/response
  util/SessionManager.kt    # SharedPreferences (token, url, deposito, etc.)
docs/                       # documentacao PDF
README.md

```

Autenticacao e Controle de Sessao

Login e JWT de sessao

POST /auth/login recebe login e senha. O backend busca o usuario em USUARIOS e tenta autenticar em ordem: primeiro compara com SENHAMOBILE (hash bcrypt, coluna adicionada pelo Invec); se nao existir, tenta SENHA (senha original do Automec em texto puro - fallback de compatibilidade). Em caso de sucesso retorna um JWT HS256 assinado com JWT_SECRET (definido no .env).

Payload do token JWT de sessao:

```

{"sub": "123",           # IDUSUARIO do Automec
 "login": "joao",
 "role": "operador",     # operador | gerente | admin
 "idgrupo": 3,           # 1=admin, 2=gerente, 3=operador
 "mobile_admin": false,  # permissao de gestao do app
 "exp": 1718000000}      # expiracao: 8h a partir do login

```

Rate limiting de login

Implementado com dicionarios em memoria (sem Redis). Dois contadores independentes por IP remoto e por login tentado. Ao atingir 5 falhas em 60 segundos, bloqueia por 300 segundos. O contador e zerado apos bloqueio expirar.

Inatividade no app (TimeoutActivity)

Todas as Activities do app herdam de TimeoutActivity. Ela usa um Handler + Runnable que dispara apos 15 minutos. Qualquer toque na tela (onUserInteraction) reinicia o timer. Ao expirar, chama SessionManager.logout() e redireciona para LoginActivity com flag FLAG_ACTIVITY_NEW_TASK | FLAG_ACTIVITY_CLEAR_TASK para limpar a pilha.

Interceptor 401 no OkHttp

RetrofitClient configura um Interceptor que monitora todas as respostas. Se receber HTTP 401 (token expirado ou invalido), automaticamente limpa a sessao no SessionManager e redireciona para LoginActivity. O usuario ve a mensagem 'Sessao expirada' e pode logar novamente sem perder os dados de bipagem (que ficam no servidor).

SessionManager - SharedPreferences

Dados persistidos localmente no celular:

Chave	Conteudo
auth_token	JWT Bearer token da sessao atual
server_url	URL base do servidor (ex: http://192.168.1.31:8000/)
deposito_id	Codigo do deposito selecionado na sessao atual
deposito_nome	Nome do deposito para exibicao
device_id	UUID gerado na primeira instalacao do app - identifica o dispositivo nos logs
usuario_login	Login do usuario logado
usuario_role	Role do usuario (operador/gerente/admin)
mobile_admin	Flag se o usuario tem admin mobile

Sistema de Licenca RSA

A licenca usa criptografia assimetrica RSA 2048-bit com algoritmo JWT RS256. A chave privada (licenca_privada.pem) fica exclusivamente com a Pontual Tecnologia e e usada para assinar as licencas. A chave publica e embutida diretamente em app/licenca.py durante o build - o cliente nunca tem acesso a ela explicitamente.

Artefato	Localizacao	Distribuir ao cliente?
licenca_privada.pem	C:\Administracao\inventario-api\	NAO - nunca
gerar_licenca.py	C:\Administracao\inventario-api\	NAO - nunca
Chave publica (embutida)	Dentro do InvecServidor.exe compilado	Sim (implicitamente)
LICENSE_KEY (JWT)	C:\Invec\env no servidor do cliente	Sim - via instalador

Payload da licenca JWT:

```
{
  "produto": "Invec",
  "cliente": "Nome da Empresa Ltda",
  "cnpj": "00.000.000/0001-00",
  "emitida_em": "2026-06-19",
  "expira_em": ""} # vazio = licenca permanente
```

Validacao no startup: main.py chama validar_licenca() no lifespan antes de aceitar qualquer requisicao. Se invalida ou expirada, sys.exit(1) encerra o processo e o NSSM registra o erro no log.

Gerar licenca para novo cliente:

```
cd C:\Administracao\inventario-api
python gerar_licenca.py
# Preenche: nome do cliente, CNPJ, validade em meses (Enter = permanente)
# Saida: LICENSE_KEY=eyJhbGciOiJSUzI1NiJ9...
```

Banco de Dados Firebird

Tabelas do Automec utilizadas (somente leitura e insercao controlada)

Tabela	Operacao	Observacao
USUARIOS	SELECT, UPDATE	Colunas SENHAMOBILE e MOBILE_ADMIN adicionadas pelo Invec via migration
DEPOSITO	SELECT	Listagem de depositos para selecao no app
PRODUTO	SELECT	Descricao, unidade e codigo do produto
PRODUTO_CODBARRA	SELECT	Tabela de codigos de barras alternativos (JOIN com PRODUTO)
MOVIMENTO	SELECT	QTDEATUAL: quantidade atual em estoque por produto e deposito
MOV_PRODUTO	INSERT, SELECT	Consolidacao grava aqui com TIPOMOVIMENTO=5. Trigger do Automec atualiza MOVIMENTO.QTDEATUAL.
SAIDAPRODUTO	SELECT	Pedidos de saida pendentes para CE (Considerar Entrega)
SAIDAESTOQUE	SELECT	Detalhe dos itens dos pedidos de saida
PRODUTOPRECO	SELECT	FATORCONV e VLCUSTO para calcular QTENTRADA/QTSAIDA no MOV_PRODUTO

Tabelas criadas pelo Invec (DDL automatico no startup)

Tabela / Coluna	Definicao e proposito
INVENTARIO_TEMP	Sessao de contagem em andamento. Colunas: CDDEPOSITO, CDPRODUTO, PRODUTO, QTDE, QTDEATUAL_SNAP (snapshot no 1o scan), OPERADOR, DATA_HORA.
OPERADORES_APP	Cadastro de operadores fisicos. Colunas: ID (autoincrement), NOME, ATIVO.

Tabela / Coluna	Definicao e proposito
LOG_INVENTARIO	Auditoria completa. Colunas: ID, TIPO, CDDEPOSITO, CDPRODUTO, PRODUTO, OPERADOR, LOGIN_USUARIO, QTDE_ANTES, QTDE_DEPOIS, MOTIVO, DEVICE_ID, DATA_HORA.
USUARIOS.SENHAMOBILE	Hash bcrypt da senha mobile do usuario. NULL = usuario nao tem acesso ao app.
USUARIOS.MOBILE_ADMIN	CHAR(1) 'S'/'N' - indica se o usuario pode gerenciar outros usuarios no app.

Roles e Permissoes

Role	IDGRUPO Automec	O que pode fazer no Invec
admin	1	Tudo: bipar, ver relatorio, editar, remover, consolidar com auto-autorizacao, ver auditoria, gerenciar operadores
gerente	2	Igual ao admin, exceto gestao de usuarios mobile (se nao for mobile_admin)
operador	3	Bipar e ver relatorio. Consolidar com divergencias exige autorizacao de gerente ou admin diferente.
mobile_admin	qualquer	Flag adicional. Pode gerenciar senhas mobile e delegar admin mobile a outros usuarios.

Fluxo de Bipagem (POST /inventario/bipagem)

O endpoint recebe: cdproduto, cddeposito, operador, device_id. Executa um UPDATE atômico com RETURNING para evitar race condition:

```
UPDATE INVENTARIO_TEMP
  SET QTDE = QTDE + 1, OPERADOR = :operador
 WHERE CDPRODUTO = :prod AND CDDEPOSITO = :dep
RETURNING QTDE
```

- Se UPDATE retornar 1 linha: produto ja existe na sessao - quantidade incrementada
- Se UPDATE retornar 0 linhas: primeiro scan do produto - executa INSERT com QTDEATUAL_SNAP = estoque atual do MOVIMENTO
- QTDEATUAL_SNAP e gravado apenas uma vez (no primeiro scan) e nao e alterado depois
- Se $QTDE > 2 \times QTDEATUAL_SNAP$ e $QTDEATUAL_SNAP \geq 10$: grava ALERTA no LOG_INVENTARIO e retorna flag alerta=true para o app mostrar aviso
- Se produto foi previamente excluido na sessao e re-escaneado: grava ALERTA_REESCAN no log

Implementacao do Scanner Android

Modo Camera (CameraX + ML Kit)

ScannerActivity usa CameraX ImageAnalysis com um BarcodeScanner do ML Kit configurado para os formatos: EAN-13, EAN-8, Code 128, Code 39, QR Code, ITF, Codabar. A análise roda em background thread via ExecutorService. Ao detectar um barcode, o resultado é postado no MainThread via runOnUiThread antes de chamar o endpoint.

- Modo simples (switch Multiplos desativado): camera fica pausada apos cada scan, reativa ao tocar em Escanear
- Modo multiplos (switch ativo): camera permanece analisando continuamente com debounce de 1.5s por código
- Preview da camera exibido em PreviewView com AspectRatio.RATIO_16_9

Modo Bluetooth (HID Keyboard emulation)

Leitores Bluetooth operam em modo HID (Human Interface Device), emulando um teclado. O app usa um EditText invisível com requestFocus() para capturar o input. Um TextWatcher detecta quando o leitor envia o código completo (terminado com \n ou \r) e dispara o scan automaticamente sem interação do usuário.

Fluxo de Consolidacao (POST /inventario/consolidar)

- Busca todos os itens de INVENTARIO_TEMP para o depósito
- Consulta SAIDAPRODUTO + SAIDAESTOQUE para CE (Considerar Entrega). Envolvido em try/except para compatibilidade com versoes do Automec sem essas tabelas
- Calcula divergencias: item diverge se $|qtde_contada - (qtdeatual + qtdeentrega)| > 0$
- Se pct_divergencias $\geq 30\%$ E total_itens ≥ 5 : retorna HTTP 422 'Recontagem obrigatoria'
- Se ha divergencias: valida credenciais do supervisor (POST /auth/login interno com role $\geq$ gerente, diferente do operador se for operador)
- Adquire lock: _consolidando_lock (threading.Lock) + _consolidando_depositos (set). Retorna 409 se o depósito já está sendo consolidado
- Gera IDINVENTARIO unico via GEN_ID(GEN_MOV_PRODUTO, 1)
- Para cada item: INSERT em MOV_PRODUTO com TIPOMOVIMENTO=5, calculando QTENTRADA/QTSAIDA conforme logica CE
- O trigger TG_INSERT_MOV_PRODUTO do Automec atualiza MOVIMENTO.QTDEATUAL automaticamente
- DELETE FROM INVENTARIO_TEMP para o depósito na mesma transação
- Grava CONSOLIDACAO no LOG_INVENTARIO e salva relatorio .txt em C:\Invec\relatorios\
- Libera o lock

Considerar Entrega (CE) - Logica de Calculo

Pedidos de saída faturados mas ainda não entregues representam estoque fisicamente presente mas já comprometido. O Invec considera essa quantidade tanto no cálculo de divergencias quanto na geração do MOV_PRODUTO (replica a logica GravarMov_Produto_Entrega do Automec):

Cenario	QTENTRADA	QTSAIDA	VL_PERDA_GANHO
Com entrega pendente e contado > atual	contado - atual	qtdeentrega	NULL (Automec calcula)
Com entrega pendente e contado < atual	0	(atual - contado) + qtdeentrega	NULL
Com entrega pendente e contado = atual	0	qtdeentrega	NULL
Sem entrega e contado > atual (ganho)	contado - atual	0	(contado - atual) x vlcusto
Sem entrega e contado < atual (perda)	0	atual - contado	(contado - atual) x vlcusto (negativo)
Sem entrega e contado = atual	0	0	0 (registro de auditoria apenas)

vlcusto vem de PRODUTOPRECO.VLCUSTO / PRODUTOPRECO.FATORCONV para o produto. A consulta CE e envolvida em try/except para degradar graciosamente em versoes antigas do Automec que nao possuem as tabelas SAIDAPRODUTO ou SAIDAESTOQUE.

Seguranca e Auditoria

Mecanismo	Implementacao
Rate limit	5 falhas em 60s -> bloqueio 300s. Dicts em memoria: {ip: [timestamps]} e {login: [timestamps]}. Nao persiste entre reinicializacoes do servidor.
JWT_SECRET	String arbitraria de minimo 32 caracteres definida no .env. Assina todos os tokens de sessao HS256.
Supervisor obrigatorio	Consolidacao com divergencias requer login/senha de gerente ou admin via endpoint de autenticacao interno. Operadores nao podem autorizar a si mesmos.
EDICAO_SUSPEITA	Edicao detectada como suspeita quando qtde_nova == qtdeatual_snap (contagem passou a coincidir exatamente com o estoque do sistema).
ALERTA_REESCAN	Se produto tem registro de EXCLUSAO no LOG_INVENTARIO nas ultimas 12h para o mesmo deposito e o mesmo device_id re-escaneia: grava ALERTA_REESCAN.
ALERTA de quantidade	qtde_nova > 2 * qtdeatual_snap AND qtdeatual_snap >= 10: grava ALERTA e retorna flag para o app exibir confirmacao.
device_id	UUID v4 gerado na primeira abertura do app, salvo em SharedPreferences. Gravado em todos os eventos do LOG_INVENTARIO para rastreabilidade por dispositivo.
Retencao de logs	run_migrations() remove: LOG com TIPO=BIPAGEM com mais de 90 dias; demais logs com mais de 365 dias; EDICAO_SUSPEITA e ALERTA_REESCAN mantidos permanentemente.

Endpoints da API

Metodo	Rota	Descricao	Acesso minimo
POST	/auth/login	Autenticacao com rate limit. Retorna JWT.	Publico
GET	/auth/usuarios	Lista usuarios do Automec com flags mobile.	mobile_admin
PUT	/auth/usuarios/{id}/senha-mobile	Define ou altera senha mobile (hash bcrypt).	mobile_admin
PUT	/auth/usuarios/{id}/toggle-admin	Ativa/desativa mobile_admin. So usuario MI pode executar.	Usuario MI
GET	/depositos	Lista depositos do Automec.	Autenticado
GET	/produtos/barcode/{codigo}	Busca produto pelo codigo de barras (PRODUTO_CODBARRA e PRODUTO).	Autenticado
GET	/produtos/busca?q=	Busca produto por descricao parcial.	Autenticado
POST	/inventario/bipagem	Registra scan atomico. Retorna qtde acumulada e flag de alerta.	Autenticado
GET	/inventario/relatorio/{dep}	Lista itens de INVENTARIO_TEMP para o deposito.	Autenticado
GET	/inventario/resumo/{dep}	Lista produtos com estoque > 0 que ainda nao foram bipados.	Autenticado
PUT	/inventario/bipagem/{id}	Edita quantidade de um item. Grava EDICAO ou EDICAO_SUSPEITA no log.	Autenticado
DELETE	/inventario/bipagem/{id}	Remove item da contagem. Grava EXCLUSAO no log.	Autenticado
POST	/inventario/consolidar	Consolida no Automec com validacao de divergencias e supervisor.	Autenticado
GET	/inventario/historico/{dep}	Lista consolidacoes anteriores (MOV_PRODUTO com TIPOMOVIMENTO=5).	Autenticado
GET	/inventario/log/{dep}	Log de auditoria do deposito (LOG_INVENTARIO).	Gerente/Admin
GET	/operadores	Lista operadores cadastrados em OPERADORES_APP.	Autenticado
POST	/operadores	Cria novo operador.	Gerente/Admin
PUT	/operadores/{id}/toggle	Ativa ou desativa um operador.	Gerente/Admin

Migrations Automaticas (run_migrations)

Executadas no lifespan do FastAPI antes de aceitar requisicoes. Cada operacao e idempotente: verifica se ja existe antes de criar/alterar.

- CREATE TABLE INVENTARIO_TEMP se nao existir
- ALTER TABLE INVENTARIO_TEMP ADD COLUMN OPERADOR se nao existir
- ALTER TABLE INVENTARIO_TEMP ADD COLUMN QTDEATUAL_SNAP se nao existir
- CREATE TABLE OPERADORES_APP se nao existir
- ALTER TABLE USUARIOS ADD COLUMN SENHAMOBILE se nao existir
- ALTER TABLE USUARIOS ADD COLUMN MOBILE_ADMIN CHAR(1) DEFAULT 'N' se nao existir
- CREATE TABLE LOG_INVENTARIO se nao existir
- CREATE INDEX IDX_LOG_INV_DEPOSITO, IDX_LOG_INV_DATA, IDX_INVTEMP_PROD_DEP, IDX_LOG_INV_TIPO
- DELETE logs expirados conforme politica de retencao
- DELETE arquivos .txt em C:\Invec\relatorios\ com mais de 180 dias

Build e Distribuicao

Servidor Windows

```
cd C:\Administracao\inventario-api
```

```
# 1. Compilar o servidor
```

```
pyinstaller servidor.spec --clean --noconfirm
```

```
# -> dist/InvecServidor.exe (~21 MB, single-file)
```

```
# 2. Compilar o instalador (bundla InvecServidor.exe + nssm.exe)
```

```
pyinstaller instalador.spec --clean --noconfirm
```

```
# -> dist/Instalar-Invec.exe (~32 MB, entrega ao cliente)
```

O instalador.spec usa datas=[(InvecServidor.exe, '.'), (nssm.exe, '.')] para incluir os binarios. O cliente recebe apenas Instalar-Invec.exe - sem Python nem dependencias.

Gerar nova licenca para cliente

```
python gerar_licenca.py
```

```
# -> imprime: LICENSE_KEY=eyJhbGciOiJSUzI1NiJ9...
```

```
# Enviar esta string ao cliente para colar no campo do instalador
```

App Android

```
cd C:\Administracao\inventario-app
```

```
.\gradlew assembleRelease
```

```
# -> app/build/outputs/apk/release/app-release.apk
```

```
# APK assinado com keystore em app/release/invec-app.jks
```

```
# Configuracoes de assinatura em app/build.gradle (signingConfigs)
```

O build usa R8 (ProGuard) para minificacao e ofuscacao do bytecode. A keystore NAO e commitada no git.